OPENSPEC / FIELD GUIDE

OpenSpec 中文实战指南

从想法到可信交付

快速入门 / 完整实战 / 团队进阶

安装 · 工作流 · CLI · OPSX · 规格与架构

以可审查的 Markdown 计划连接人、AI 编码助手与代码。

面向新用户快速上手，也为团队提供配置、评审、自动化与跨仓库规划的完整参考。

版本基线 v1.15.0 | 官方资料核对 2026-10-28

macOS / Linux / Windows

独立中文教程 · 案例与图表为教学整理

按你的目标开始阅读

路线	内容与起始页
第一次使用	03 核心概念 → 04 安装 → 06 初始化 → 07 全流程
已经安装	06 项目接入 → 10 /opsx 指令 → 13 实战
每天查阅	18 CLI 设置 → 19 浏览与校验 → 20 工作流 → 22 工具命令
团队推广	23 项目配置 → 24 Schema → 25 团队协作 → 26 跨仓库
遇到问题	29 环境与指令排错 → 30 规格与归档排错

章节索引

03 核心概念
04 多平台独立安装
05 包管理器与升级
06 初始化与目录
07 全流程图
08 计划产物与依赖
09 规格写法
10 默认六个 /opsx 工作流
11 扩展六个 /opsx 工作流
12 工具调用方式对照
13 需求与提案
14 可验收的 Delta 规格
15 设计、任务与最小代码
16 实现、测试与归档
17 修 Bug 与纯重构
18 设置与版本命令
19 查看、校验、归档
20 产物工作流命令
21 配置与 Schema 子命令
22 工具命令与旧入口
23 项目配置
24 自定义 Schema
25 分支、PR 与验收
26 Stores：跨仓库规划（Beta）
27 References、Context 与 Worksets
28 架构与执行链路
29 环境与指令
30 规格、状态与归档
31 最佳实践与 CI
32 官方来源与版本说明
33 延伸阅读、术语与许可
34 上游 MIT License

最短路线：初始化 → propose → 人工审查 → apply → 测试 → archive。先掌握这个闭环，再启用扩展工作流。本文中标注“团队建议”的规则需要你用评审或 CI 落实。

OpenSpec 到底解决什么问题

只有聊天记录时，需求容易在多轮对话、换模型或换同事后丢失。OpenSpec 将计划写入仓库，让 AI 每次都能重新读取同一份目标与进度。它兼容新项目，也适合在已有代码库上做增量改动。[R1][R4]

概念	中文理解	主要位置
Spec	对当前系统行为的长期约定	openspec/specs/
Capability	一组内聚的系统能力，如 user-auth	specs/<能力路径>/spec.md
Change	一次有明确意图的变更	openspec/changes/<变更名>/
Delta spec	这次变更对规格的增量修改	changes/<变更名>/specs/
Artifact	计划产物：提案、规格、设计、任务等	变更文件夹内
Schema	产物、模板及其依赖关系的定义	内置或 openspec/schemas/
Profile	本机选择安装哪些助手工作流	全局 config.json
Archive	合并规格并保存变更历史	changes/archive/

两类入口，职责不同

终端里的 openspec init、validate、status 负责设置、结构检查与状态查询。AI 聊天里的 /opsx:propose、apply 等指令负责理解需求、编写文档或代码。CLI 不会因你运行 new change 就自动调用模型生成完整提案。

三个必须分清的“完成”

产物文件已存在，表示计划具备继续条件；tasks.md 勾选完，表示助手记录了执行进度；代码满足需求，需要测试和评审给出证据。三者应相互核对。OpenSpec 不替代编译器、测试框架、版本管理或发布系统。

> 判断一段内容应该放哪里：为什么改 → proposal；对外行为 → specs；如何实现 → design；具体执行与验证 → tasks。

首选：无需 Node.js 的安装方式

v1.15.0 提供独立可执行文件。安装脚本按系统和 CPU 架构下载发布资产，并比对 SHA256SUMS。你已安装时，先运行 openspec --version，无需重复安装。[R2][R3]

macOS / Linux：终端

```
curl -fsSL https://raw.githubusercontent.com/Fission-AI/OpenSpec/main/scripts/binary/install.sh | sh
openspec --version
```

默认位置是 ~/.local/bin。若安装提示 PATH 未包含此目录，按其提示加入后重新打开终端。Alpine 先安装运行库：apk add libstdc++ libgcc。

Windows：PowerShell

```
irm https://raw.githubusercontent.com/Fission-AI/OpenSpec/main/scripts/binary/install.ps1 | iex
openspec --version
```

默认目录为 %LOCALAPPDATA%\Programs\openspec，脚本将其加入用户 PATH。安装后旧终端找不到命令时，先新开一个 PowerShell 窗口。

平台与最低要求	手动下载资产
macOS 13+，Apple silicon / Intel	openspec-darwin-arm64 / openspec-darwin-x64
Linux glibc 2.17+，arm64 / x64	openspec-linux-arm64 / openspec-linux-x64
Alpine 3.17+，arm64 / x64	openspec-linux-arm64-musl / openspec-linux-x64-musl
Windows 10+，arm64 / x64	openspec-windows-arm64.exe / openspec-windows-x64.exe

手动安装：从官方 Releases 选对应文件，改名为 openspec（Windows 为 openspec.exe），放到 PATH 中。macOS/Linux 还需 chmod +x openspec。浏览器下载的 macOS 文件可能需要 xattr -d com.apple.quarantine openspec；只对已确认来源的下载操作。

> 脚本安装会执行远程代码。希望先审阅时，下载官方脚本检查后再运行；或采用手动下载与校验。可选构建来源验证：gh attestation verify <资产文件> --repo Fission-AI/OpenSpec。

沿用熟悉的包管理器也可以

Node 包安装方式要求 Node.js ≥ 20.19.0。Homebrew 与 Nix 会处理它们自己的依赖。Bun 全局安装 Node CLI 后仍需要系统中的 Node；现代 Yarn 不再提供 global add。[R3]

方式	安装命令	注意
npm	npm install -g @fission-ai/openspec@latest	先用 node --version 检查环境
pnpm	pnpm add -g @fission-ai/openspec@latest	需配置全局可执行目录
Yarn Classic 1.x	yarn global add @fission-ai/openspec@latest	Yarn 2+ 改用其他方案
Bun	bun add -g @fission-ai/openspec@latest	运行仍依赖 Node
Homebrew	brew install openspec	macOS/Linux；无需额外 tap
Nix	nix profile install github:Fission-AI/OpenSpec	项目提供官方 flake

Deno 属于进阶可选安装，需要授权读写、环境与网络等权限；使用官方安装页的完整命令，避免漏掉权限。一次性体验 Nix：nix run github:Fission-AI/OpenSpec -- --version。

升级分成两个层次

```
# 任选与你安装来源一致的一条
openspec install
npm install -g @fission-ai/openspec@latest
brew upgrade openspec

# 然后在每个项目中刷新助手指令
openspec update --dry-run
openspec update
```

openspec install 用于安装/升级独立版本；npm、Homebrew 用户执行它会切换安装来源。项目的生成指令不会随 CLI 自动刷新。update 在交互环境可提示新版本，但自动升级能力取决于安装来源。

避免多个副本抢占 PATH

macOS/Linux 用 which -a openspec，Windows 用 where.exe openspec。openspec version 可显示当前安装来源。nvm 切换 Node 后，原全局 npm 安装可能不再可见。

卸载前先 openspec completion uninstall，再使用对应包管理器卸载或删除独立文件。openspec/specs/ 和 archive/ 是项目知识资产，应按团队保留策略处理。

在项目根目录接入 OpenSpec

CLI 每台机器安装一次；init 则按项目运行。已有项目先确认当前路径与版本控制状态，再选择实际使用的助手，避免生成一堆无用集成文件。[R5][R8]

```
cd your-project
openspec init

# 自动化或明确知道工具时
openspec init --tools codex --language Chinese
# 同一项目同时支持两种工具
openspec init --tools claude,cursor
```

--language 用于新建项目配置；已有配置通过 context 调整语言。无交互运行请明确 --tools；--tools none 只设置配置，不安装助手集成。

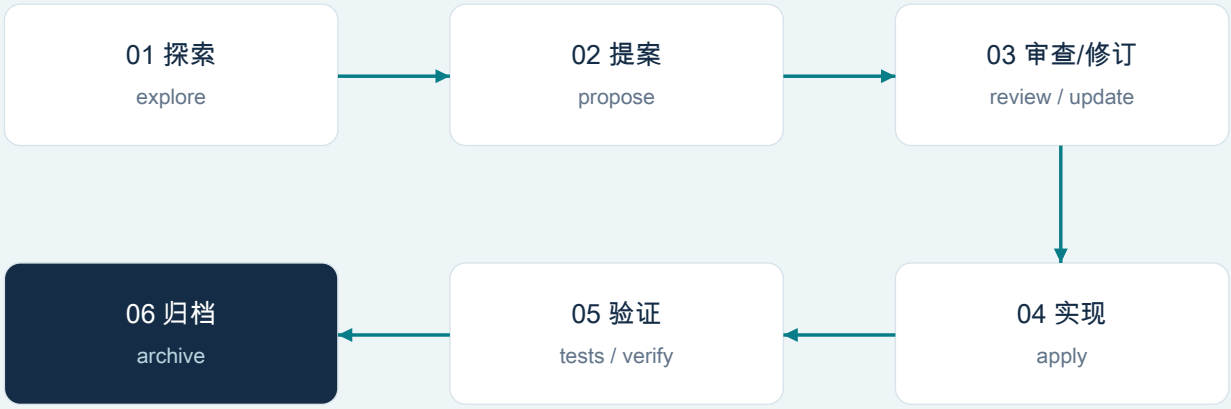
```
your-project/
  src/                          业务代码
  openspec/
    config.yaml                   项目上下文与规则
    specs/                      当前系统规格，按需出现
    changes/                    活跃变更，按需出现
      add-dark-mode/
        .openspec.yaml        变更元数据
        proposal.md
        specs/theme/spec.md
        design.md             按需要生成
        tasks.md
      archive/                已完成变更，按需出现
  .agents/skills/openspec-*/        选择 Codex 后的集成
```

第一次初始化后的检查

1. 确认 openspec/config.yaml 存在；空项目中的 specs/、changes/ 可以尚未出现。
2. 看 init 结尾的 Getting started 提示，使用它给出的准确调用方式。
3. 重启或重新加载 AI 工具，确认技能/命令可见。
4. 将项目上下文、能力规格和变更纳入版本管理。生成集成文件的提交策略由团队统一。

> 不要把个性化规则直接改进生成的 SKILL.md。它们可能被 update 覆盖；团队约束放 config.yaml、AGENTS.md 或项目文档。

一条从想法到归档的闭环



发现新信息时回到计划修订；归档后进入下一次变更。

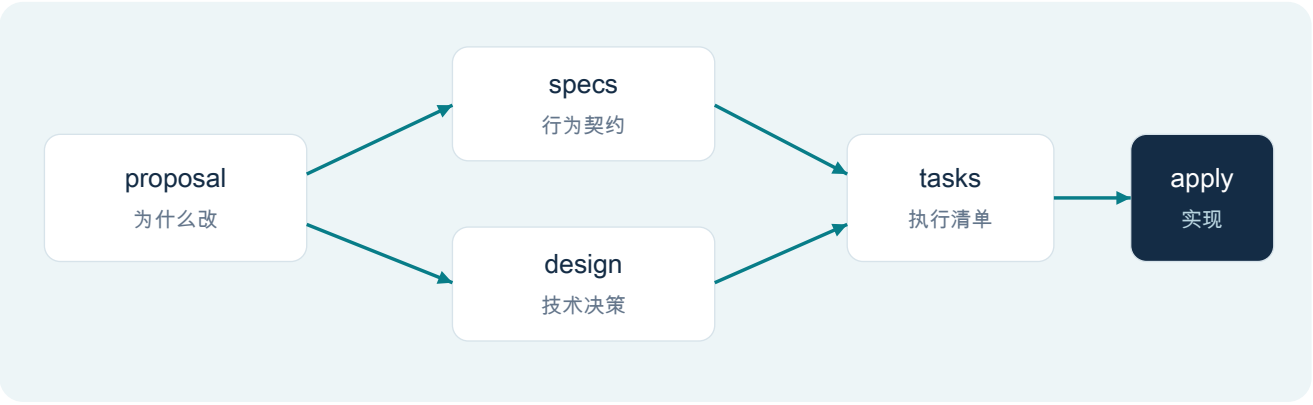
每一步你要判断什么

步骤	应得到什么	人的检查点
Explore	问题、边界、候选方案	是否已有能力？需求是否明确？
Propose	可审查的变更计划	规格是否定义了真正可接受的结果？
Review / Update	一致的提案、规格、设计和任务	是否遗漏异常、兼容和迁移？
Apply	代码、测试、任务进度	每项完成是否有实际证据？
Verify	结构检查、行为测试、评审意见	是否满足需求与设计？
Archive	当前主规格 + 带日期的历史变更	同步内容是否真实反映交付？

可在任意阶段回到计划修订。需求已经明确时可以从 propose 开始。verify 为可选助手工作流，测试与评审仍应按项目需要执行。[R4][R6]

> 归档不是发布，也不会替你合并 PR。Git、CI 与部署由已有工程流程负责。

四份文档各司其职



proposal 是起点；specs 和 design 都依赖 proposal；tasks 汇合它们；apply 读取 tasks。图表示默认 schema 的依赖，两个就绪产物不必同时生成。[R9]

文件	写什么	审查问题
proposal.md	Why、What Changes、Capabilities、Impact	意图单一吗？能力路径准确吗？
specs/<能力>/spec.md	行为增量、需求、场景	测试人员能据此判定通过吗？
design.md	决策、备选方案、风险、迁移	方案与代码库约束一致吗？
tasks.md	小而可验证的复选框任务	规格中的行为是否都有任务覆盖？

“可省略”需要依据

简单变更没有架构、依赖、数据模型等设计决策时，默认流程可按条件省略 design。纯重构、工具或文档变更如果没有行为变化，可在 .openspec.yaml 声明 skip_specs: true，明确省略 specs。不要为了凑齐文件虚构需求。

文件进度不等于质量认证

status 根据实际文件与 schema 推导 ready、blocked、done 等状态；它不会理解一篇规格是否正确。JSON 中 isPlanningComplete 表示计划产物齐备，isComplete 是兼容别名，二者都不代表任务已实现。任务完成应查看 tasks 或 instructions apply 的任务状态。

何时拆分变更

一个变更应该能用一句话说明意图。登录改造、报表导出与主题切换通常应拆开；某个需求横跨前后端但必须一起验收时，可以保留为同一变更并明确接口边界。

将“好用”改写为可验证的行为

保留 OpenSpec 识别的英文标题与关键字，标题后的名称和正文可以用中文。Requirement 是需求，Scenario 是验证该需求的具体情境；场景标题必须是四级标题。[R9][R10]

```
## ADDED Requirements

### Requirement: 保存主题偏好
系统 SHALL 在浏览器存储可用时保留用户选择的主题。

#### Scenario: 刷新后保留深色
- **GIVEN** 用户已选择深色且存储成功
- **WHEN** 用户刷新页面
- **THEN** 页面仍显示深色
```

Delta 标题	用途	最容易犯的错
ADDED Requirements	新需求	把已有需求的修改当成新增
MODIFIED Requirements	替换既有需求	只写修改的一句话，丢失旧场景
REMOVED Requirements	删除需求	未写 Reason 与 Migration
RENAMED Requirements	仅重命名	旧名不匹配或同时改了行为

```
## RENAMED Requirements
- FROM: `### Requirement: Theme selection`
- TO: `### Requirement: 主题选择`
```

MODIFIED 应复制现有需求的完整块，包括全部场景，再编辑；名称要与主规格对应。重命名且修改行为时，需要配合 MODIFIED，而不能只使用 RENAMED。

写强规格的四个检查

1. 用 SHALL / MUST 表达必须满足的行为，避免“尽量”“合理”等模糊措辞。
2. 给出输入、输出、错误和边界条件，不把内部类名或框架选择写成外部需求。
3. 每个需求至少有一个真正检验它的场景；重要需求同时覆盖异常路径。
4. 新能力写清 Purpose，避免归档后留下 TBD。Purpose 建议至少 50 字符，需求描述尽量不超过 500 字符，详细例子放场景。

在 AI 聊天中输入这些指令

本页使用官方通用写法。Codex 等工具请换成第 12 页的技能名。终端无法直接执行 /opsx:propose。[R6][R8]

指令	中文解释	示例
/opsx:explore [主题]	读取项目、调查问题、比较方案；不写代码。默认不写文件，明确要求记录时可写计划	/opsx:explore 深色模式如何接入
/opsx:propose [名称或描述]	建立变更并按依赖生成计划；到可审查阶段停止	/opsx:propose add-dark-mode
/opsx:apply [变更名]	读取计划并实施未完成任务，记录复选框进度；遇阻塞暂停	/opsx:apply add-dark-mode
/opsx:update [变更名]	修改已有计划，协调文档一致性；逐项确认修改，不改代码	/opsx:update add-dark-mode
/opsx:sync [变更名]	将 delta 合并到主规格；变更仍保持活跃	/opsx:sync add-dark-mode
/opsx:archive [变更名]	检查完成度、处理待同步规格并将变更移到历史目录	/opsx:archive add-dark-mode

一次典型的需求调整

```
/opsx:update add-dark-mode
新增约束：浏览器禁用持久化存储时仍应允许切换，
但允许刷新后丢失选择。请同步检查规格、设计和任务。
```

批准修订后再 /opsx:apply add-dark-mode，使代码跟上新计划。update 主要维护已存在的产物；完全未开始的产物由 continue 或 CLI instructions 引导生成。

两种 update、两种 archive

openspec update 刷新工具集成；/opsx:update 修订变更计划。

openspec archive 是终端中的解析、校验与归档命令；/opsx:archive 是助手流程，包含解释、检查和语义同步交互。不要将它们当成相同入口的不同拼写。

> 有多个活跃变更时显式写名称。归档前确认未完成事项与同步目标，不把助手的“已完成”当成测试结果。

需要更细控制时再启用

先在终端运行 openspec config profile，在选择器中添加所需工作流，再在项目内 openspec update。默认 core 并不安装下表的六项。[R6][R7]

指令	中文解释	适合何时
/opsx:new [名称]	只建立变更骨架与元数据，展示第一份模板，不生成完整计划	复杂需求需要逐份审查
/opsx:continue [名称]	按依赖创建下一份就绪产物，一次一份	每一步都要控制细节
/opsx:ff [名称或描述]	按依赖补齐实现所需的计划产物；不写业务代码	需求明确、希望快速规划
/opsx:verify [名称]	只读核对实现与计划的完整性、正确性、一致性并给报告	归档前检查遗漏与偏离
/opsx:bulk-archive	选择多个变更，检查同步冲突并批量归档	一批并行工作已经结束
/opsx:onboard	用实际代码库走一遍变更到归档的教学流程，会产生真实文件	希望边做边学

两条计划路线

```
快速规划：propose → 审查 → apply
逐份规划：new → continue → continue → … → 审查 → apply
```

ff 与 propose 都有快速生成计划的用途。新手用 propose 即可；启用逐份路线后，按已安装版本的提示使用 continue 或 ff。已有变更优先 continue/update，避免重复建立目录。

看懂 verify 报告

Completeness 检查任务、需求和场景是否覆盖；Correctness 检查实际行为是否符合意图；Coherence 检查设计与实现是否一致。CRITICAL 优先解决，WARNING 需要判断和解释，SUGGESTION 是改进建议。

> verify 为只读报告，不自动修复、不运行发布、不保证归档被阻止。需要硬性门禁时，在评审规则和 CI 中明确执行。

同一个工作流，不同工具的拼写

以 init 最后输出的提示与工具中的技能选择器为准。下面给出常见集成，完整工具列表见参考资料。[R8]

工具	创建提案	实施变更
Claude Code / Gemini CLI	/opsx:propose	/opsx:apply
Cursor / Copilot IDE	/opsx-propose	/opsx-apply
Antigravity / OpenCode	/opsx-propose	/opsx-apply
Amazon Q	@opsx-propose	@opsx-apply
Codex	$openspec-propose	$openspec-apply-change
Kimi Code	/skill:openspec-propose	/skill:openspec-apply-change
仅技能的兼容工具	/openspec-propose	/openspec-apply-change

Codex 中十二项工作流的准确技能名

通用工作流	Codex 调用	通用工作流	Codex 调用
explore	$openspec-explore	propose	$openspec-propose
apply	$openspec-apply-change	update	$openspec-update-change
sync	$openspec-sync-specs	archive	$openspec-archive-change
new	$openspec-new-change	continue	$openspec-continue-change
ff	$openspec-ff-change	verify	$openspec-verify-change
bulk-archive	$openspec-bulk-archive-change	onboard	$openspec-onboard

Codex 使用 .agents/skills/openspec-*/SKILL.md；即使 delivery 为 commands，Codex 集成仍使用 skills。不要照搬旧教程去找全局 ~/.codex/prompts/opsx-*.md。

工具边界

Copilot IDE 的 prompt 文件与 Copilot CLI、云端 coding agent 是不同集成。云端配置需单独选择，会生成 GitHub Actions 等文件。某些纯技能工具依靠自然语言匹配技能，并非都把 /openspec-* 注册为可输入命令；找不到时明确要求“使用 openspec-propose 技能”。

为现有网页增加深色模式

案例假设：已有可以启动的网页项目，本书不限定 React、Vue 或原生 HTML。先在工作分支或可回退的目录上试做。下面的文档片段是你应当审查的目标产物，并非承诺助手每次输出完全相同。[R4][R10]

第一步：调查现状

```
/opsx:explore
检查项目现有主题、样式入口和测试方式。
我要增加浅色/深色切换。请先比较最小改动方案，
说明刷新、系统主题变化、存储异常会如何处理。
```

确定本次边界：初次打开随系统；手动选择优先并在可用时持久化；存储失败时仍可切换。暂不增加第三个“跟随系统”按钮，不做多端账户同步。

第二步：提出变更

```
/opsx:propose add-dark-mode
沿用现有技术栈。按刚才确认的边界生成计划，
保留英文结构标题，中文编写正文，并列出测试方式。
```

proposal.md 的可用示例

```
## Why
夜间使用时界面过亮，需要可持续生效的深色模式。

## What Changes
- 首次加载遵循系统主题。
- 提供主题切换；保存手动选择，存储失败时降级。
- 覆盖现有主要页面和键盘操作。

## Capabilities
### New Capabilities
- theme: 主题选择、偏好保留和系统主题响应
### Modified Capabilities
无。

## Impact
影响全局颜色变量、页面启动逻辑、主题按钮与测试。
不改变后端接口，不增加账户级主题同步。
```

先把边界写成场景

保存到 openspec/changes/add-dark-mode/specs/theme/spec.md。新能力 Purpose 描述其长期职责；Delta 使用 ADDED。已有 theme 能力时应先读取主规格，再判断哪些内容属于 MODIFIED。

```
## Purpose
定义网页的主题选择与偏好保留行为，使用户在不同
系统主题和浏览器存储条件下获得一致、可解释的显示
结果，并为后续主题相关功能提供稳定的行为约定。

## ADDED Requirements

### Requirement: 初始主题
没有有效手动偏好时，页面 SHALL 使用系统主题。
#### Scenario: 首次访问且系统为深色
- **GIVEN** 没有已保存的有效偏好
- **WHEN** 系统偏好为深色且页面加载
- **THEN** 页面使用深色
#### Scenario: 系统主题变化
- **GIVEN** 本次会话没有手动选择且无有效已存偏好
- **WHEN** 系统主题从浅色变为深色
- **THEN** 页面切换到深色

### Requirement: 手动主题选择
用户 SHALL 能切换主题，且手动选择优先于系统主题。
#### Scenario: 手动浅色优先
- **GIVEN** 用户已选择浅色
- **WHEN** 系统切换为深色
- **THEN** 页面仍为浅色

### Requirement: 主题偏好持久化
系统 SHALL 尝试保存手动选择，存储失败不得阻止切换。
#### Scenario: 刷新保留选择
- **GIVEN** 深色选择已成功保存
- **WHEN** 用户刷新页面
- **THEN** 页面仍为深色
#### Scenario: 浏览器禁止存储
- **GIVEN** 读写存储会抛出异常
- **WHEN** 用户选择深色
- **THEN** 页面立即变深且功能继续可用
- **AND** 本会话中不被系统主题变化覆盖
```

> 按本例补充：非法存储值视为无有效偏好；按钮必须可用键盘触发且有可理解名称。这些应写进正式规格，避免仅存在于聊天中。

让规格落到清晰的技术决策

本例采用 CSS 自定义属性 + 根元素 data-theme + localStorage。理由：小型网页无需新依赖，可复用既有样式入口。系统偏好由 matchMedia 提供；读写存储均捕获异常。复杂组件库项目优先使用其已有主题方案。

```
## 1. 主题决策
- [ ] 1.1 实现偏好解析与主题决策，测试有效值、非法值和系统回退
- [ ] 1.2 接入启动和系统主题监听，验证无手动选择时跟随系统

## 2. 交互与持久化
- [ ] 2.1 按钮支持键盘与可访问名称，验证切换后颜色和名称
- [ ] 2.2 存储异常时保留会话选择，测试刷新和异常路径
- [ ] 2.3 更新主要页面色值，逐页检查文字、边框和焦点可见性

## Workflow follow-up
- 评审、合并后按团队约定同步规格并归档。
```

可直接运行的决策函数：theme.mjs

```
export function resolveTheme(saved, systemDark) {
    if (saved === "light" || saved === "dark") {
        return saved;
    }
    return systemDark ? "dark" : "light";
}

export function saveTheme(storage, theme) {
    try {
        storage.setItem("theme", theme);
        return true;
    } catch {
        return false;
    }
}
```

这只实现决策与写入降级。完整网页还需要安全读取、会话内手动偏好、系统事件监听、data-theme 和按钮集成；将它们交给 apply 按任务完成。不能用这两个函数的测试代替页面验收。

把“完成”落实到可检查的结果

在新会话调用 /opsx:apply add-dark-mode；要求助手先读取变更目录和项目约束。下面是 theme.test.mjs，可与上一页函数放在同一目录，以 node --test 运行（Node ≥ 20.19.0）。

```
import test from "node:test";
import assert from "node:assert/strict";
import { resolveTheme, saveTheme } from "./theme.mjs";

test("手动偏好优先", () => {
  assert.equal(resolveTheme("light", true), "light");
  assert.equal(resolveTheme("dark", false), "dark");
});
test("非法偏好回退到系统", () => {
  assert.equal(resolveTheme("unknown", true), "dark");
  assert.equal(resolveTheme(null, false), "light");
});
test("禁止存储时返回失败状态", () => {
  const storage = {
    setItem() {
      throw new Error("blocked");
    }
  };
  assert.equal(saveTheme(storage, "dark"), false);
});
```

页面验收	预期结果
清除偏好后首次打开	随系统；非法存储值也回退
手动选浅色后改变系统主题	保持浅色；刷新后仍保持（存储可用）
禁止存储后点击按钮	页面切换成功；本会话不被系统覆盖
键盘 Tab 与 Enter/Space	可聚焦、可触发、名称清楚
逐页检查深色主题	文字、链接、边框、焦点均可辨认

终端运行 openspec validate add-dark-mode --strict，按项目方式执行测试与构建。安装了 verify 再运行 /opsx:verify add-dark-mode。审查代码与计划一致后，用 /opsx:archive add-dark-mode 确认同步；然后检查 openspec/specs/theme/spec.md 与归档目录。

按行为变化选择文档规模

Bug 修复：回到既有承诺

假设规格已要求“存储异常时仍可切换”，但实现直接抛错。先复现，再修实现与测试；若规格已经正确，不必把同一句话再次 ADDED。若原规格确实漏掉异常约束，则建立变更补充需求和场景。[R10][R11]

```
/opsx:propose fix-theme-storage-failure
主题写入失败会阻止切换。先读取现有 theme 规格，
判断是实现违约还是规格缺口，避免重复新增需求。
```

纯重构：不改变对外行为

例如拆分主题模块但全部行为保持不变，创建变更后在 .openspec.yaml 中明确写入：

```
schema: spec-driven
skip_specs: true
```

保留提案与可验证的任务。运行 status 应看到 specs 被跳过；结构校验无需你杜撰一份“代码更清晰”的行为规格。只在确信外部行为没有变化时使用此标记。

变更已经开始，需求又改了

小范围修订：update 修订现有文档 → 审核新增/失效任务 → apply 继续。实现方案改了但行为不变，主要调整 design 与 tasks；行为变了，先调整 specs。

意图已经改变：新建变更，说明与旧变更的关系。已经归档的功能再次修改时，在当前主规格基础上新建 delta，保留历史的真实性。

取消、回滚与归档的区别

未实施变更可以按团队约定删除或记录取消原因。已部署功能回滚时，代码回退还要让当前规格恢复对应行为；宜建立新的回滚变更。archive 只是记录完成的计划，不能撤销代码或部署。

> 团队建议：不要只为“进度好看”勾选没做的任务；调整范围时明确记录取消或拆分原因，并重新核对验收。

终端命令速查：设置

以下覆盖 v1.15.0 的公开 CLI 命令组与主要可用子命令。参数以本机 openspec <命令> --help 为最终依据；尖括号是占位符，执行时替换。[R12]

命令	中文用途与主要参数
openspec --help	根帮助；help <command> 查看某命令；-h 是简写
openspec --version	输出版本，简写 -V；--no-color 关闭彩色输出
openspec version	显示安装版本/来源；--check 联网检查更新；--json 结构化输出
openspec install [version]	安装或更新独立版本；不给版本时选择最新可用发布
openspec init [path]	初始化指定项目，默认当前目录
openspec update [path]	根据本机 profile/delivery 刷新项目助手文件

init 的主要选项

--tools <id,id>：逗号分隔工具 ID，可用 all 或 none。常见 ID 为 codex、claude、cursor、gemini、github-copilot、antigravity、opencode、agents。

--language <language>：为新配置添加语言指令。--profile core|custom：本次初始化的工作流集合。

--no-animation：禁用欢迎动画。--force：不询问便清理旧版文件，应先审查现有内容。

--copilot-cloud / --no-copilot-cloud：明确启用/跳过 Copilot 云端文件设置。

update 的主要选项

--dry-run 只预览新增、修改和删除，--dry-run --json 输出机器可读计划。--force 强制重写当前生成文件。单独 --json 不受支持，应与 --dry-run 合用。

```
openspec update --dry-run --json
openspec update --dry-run --force
openspec update
```

> 升级前先确认安装来源、审查生成文件的本地修改。update 的版本检查可能提示升级 CLI；离线或不希望检查时，可设置 OPENSPEC_NO_UPDATE_CHECK=1。

终端命令速查：日常维护

命令	中文用途与主要参数
openspec list	列活跃变更；--specs 列能力；--archived 只列历史；--all 含活跃与历史；--sort recent/name；--json
openspec view	显示终端概览，便于查看规格与变更
openspec show [item]	查看变更或能力；同名时 --type change/spec；--json；--no-interactive
openspec validate [item]	检查结构与 delta；--all / --changes / --specs；--strict；--json
openspec archive [name]	校验、更新主规格、移动归档；-y/--yes 跳过确认

show 的过滤能力

变更用 --deltas-only（JSON 模式）只看增量，--diff 显示逐需求差异；规格用 --requirements 或 --no-scenarios（JSON）省去场景，-r/--requirement <序号> 查看某条需求。概要筛选后，做语义判断前仍应读完整相关规格。

```
openspec list --specs
openspec show theme --type spec
openspec show theme --type spec --json --no-scenarios
openspec validate add-dark-mode --strict --no-interactive
openspec validate --all --strict --json
openspec validate --archived
```

validate 还支持 --type、--report full/findings（批量报告详略）和 --concurrency <数量>，默认并发 6。--archived 是独立检查范围，检查历史变更中的任务是否全部完成，不重新应用或校验已同步 delta；--all 不等于包含这项检查。

archive 的边界参数

--yes 用于已确认目标和效果的非交互调用，必须明确变更名；--json 返回结构化归档结果。--skip-specs 仅跳过本次规格更新；长期没有行为 delta 的变更应使用元数据 skip_specs。--no-validate 跳过校验且需额外确认，不宜作为日常消除错误的方法。

归档通常生成 changes/archive/YYYY-MM-DD-<name>/。目标重名或 delta 不合法时应先排错。CLI 支持失败恢复保护，仍应通过版本管理检查实际结果。[R12]

这些命令多由 AI 助手使用

命令	中文用途与主要参数
openspec new change <name>	建立变更骨架；--schema；--description；--goal；--json；--store
openspec status	查产物状态；--change <id> 指定变更；--all 看全部；--json；--schema
openspec instructions [artifact]	输出模板与执行输入；--change；--json；--schema
openspec templates	查看解析后的模板路径；--schema；--json
openspec schemas	列可用 schema、来源、描述与流程；--json

手动观察一次计划生成过程

```
openspec new change add-dark-mode
openspec status --change add-dark-mode
openspec instructions proposal --change add-dark-mode
openspec instructions specs --change add-dark-mode --json
openspec instructions design --change add-dark-mode --json
openspec instructions tasks --change add-dark-mode --json
```

新建后 proposal 应就绪，下游受依赖约束。instructions 只是把“该写什么”返回给调用者；你或助手仍要按其内容编写产物。不要把输出 JSON 原样保存成 proposal.md。

apply 与 archive 也是 instructions 的输入界面

```
openspec instructions apply --change add-dark-mode --json
openspec instructions archive --change add-dark-mode --json
```

前者提供实施上下文及任务；后者只读取归档的当前 context 与 operationGuidance 等输入，不执行移动、不合并主规格。真正归档用 archive 命令或助手归档工作流。

变更名规则

使用小写字母、数字和单个连字符，如 add-dark-mode 或 100-add-auth。禁止空格、下划线、大写、连续连字符和首尾连字符。中文需求正文可以保留中文，目录名建议用可读英文。

> 自动化调用显式指定 --change、--type 与 --no-interactive（命令支持时），并检查退出码和 JSON 内容，避免脚本等着交互输入。[R12]

全局偏好与项目工作流分开管理

config 子命令	中文解释
path	显示全局 config.json 的真实位置
list [--json]	列出当前设置与 profile 概要
get <key>	读取一个键，方便脚本使用
set <key> <value>	写入值并进行类型转换；--string 强制字符串
unset <key>	删除键，恢复该键的默认行为
reset	重置配置；先用 --help 确认范围，--all --yes 重置全部
edit	用 $EDITOR 打开全局配置
profile [preset]	交互选择 delivery/工作流；preset core 恢复默认六项

全局文件：macOS/Linux 通常在 ~/.config/openspec/config.json，Windows 在 %APPDATA%\openspec\config.json。XDG_CONFIG_HOME 可影响位置，用 config path 最稳妥。当前 --scope 只支持 global，项目上下文应编辑 config.yaml。[R7][R12]

schema 子命令	中文解释与主要参数
init <name>	建项目本地 schema；--description；--artifacts；--default/--no-default；--force；--json
fork <source> [name]	复制已有 schema 到项目中；--force；--json
validate [name]	检查 schema、模板和依赖；--verbose；--json；省名检查全部
which [name]	查看 schema 解析来源；--all；--json

```
openspec config profile
openspec update
openspec schema fork spec-driven team-flow
openspec schema validate team-flow
openspec schema which team-flow
openspec new change add-search --schema team-flow
```

--force 可能覆盖文件，先检查目标目录。schema 为实验性接口，团队采用后应锁定 CLI 版本，将 schema 与模板一起纳入版本控制，升级时检验兼容性。

补全、反馈、兼容命令

命令	中文解释
openspec completion generate [shell]	向标准输出生成补全脚本
openspec completion install [shell]	安装当前或指定 shell 的补全
openspec completion uninstall [shell]	卸载补全，宜在卸载 CLI 前执行
openspec feedback <message>	提交问题反馈；可能调用浏览器/GitHub，不用于自动反馈项目隐私内容
openspec change show [name]	旧式查看变更入口，改用 show --type change
openspec change list	旧式列变更入口，改用 list
openspec change validate [name]	旧式变更检查，改用 validate --type change
openspec spec show [id]	旧式查看规格入口，改用 show --type spec
openspec spec list	旧式列规格入口，改用 list --specs
openspec spec validate [id]	旧式规格检查，改用 validate --type spec

补全支持 Bash、Zsh、Fish 与 PowerShell。安装/卸载的具体选项用该子命令 --help 查看。Nix 包已带标准位置的补全脚本，一般无需重复安装。[R3][R12]

常用环境变量

名称	中文解释
OPENSPEC_TELEMETRY=0	关闭匿名用量统计，也可关闭相关更新检查
DO_NOT_TRACK=1	关闭匿名统计的另一入口
OPENSPEC_NO_UPDATE_CHECK=1	跳过更新检查
OPENSPEC_CONCURRENCY	校验命令默认并发数量
OPENSPEC_INSTALL_DIR	独立安装脚本的目标目录
OPENSPEC_VERSION	安装脚本固定版本，如 v1.15.0

旧教程迁移提醒

旧版 /openspec:proposal、/openspec:apply、/openspec:archive 不是当前新项目的推荐入口。升级前检查旧集成，先运行 update --dry-run；不要假定旧命令会自动存在。变更文档通常仍可在 OPSX 中继续，迁移后核对本机帮助与实际生成文件。

让 AI 每次都读到必要的约束

配置放 openspec/config.yaml。schema 选择工作流；context 传达持续有效的事实；rules 为指定产物补充内容规则；operations 给 apply/archive 添加操作建议。[R13]

```
schema: spec-driven
context: |
  本项目是已有网页产品，保留现有技术栈。
  产物正文使用中文；保留 OpenSpec 英文结构标题。
  公共接口保持兼容，新增行为必须有可验收场景。
rules:
  proposal:
    - 写清本次范围与不做的内容
  specs:
    - 包含与需求相关的异常和边界场景
  design:
    - 解释关键技术选择与替代方案的取舍
  tasks:
    - 每个任务包含完成证据或验证方法
operations:
  apply:
    guidance:
      - 优先运行与当前修改相关的测试
  archive:
    guidance:
      - 汇报规格同步结果与未解决事项
```

配置	生效范围	注意
context	产物生成、apply、archive	只放必要事实，最长 50KB；过大忽略并警告
rules.<artifact>	对应产物的生成	与内置指导叠加；verify 不读取这类规则
operations.apply/archive.guidance	对应操作	是建议，不能替代强制检查
schema	默认工作流	不会批量改写已有变更的元数据

规则多了不一定更可靠

详细的永久文档保存在仓库，用简短上下文指向它们。不要把整个 README、代码库或所有历史决策塞入 context。修改配置后，下次 instructions 会重新读取；配置中的自然语言约束没有自动执行的强制能力，关键要求需工具校验。

先改项目规则，再考虑改流程

默认产物已经足够时用 config.yaml。只有确实需要新的产物、依赖或模板时才 fork schema。下面演示增加“实施前评审记录”，它是流程示例，不是自动审批系统。[R14]

```
openspec schema fork spec-driven team-flow
```

在复制得到的 schema.yaml 中，保留原产物，并在 tasks 之前加入 review，修改 tasks 的 requires（不要保留两个同名 tasks）：

```
  - id: review
    generates: review.md
    description: 实施前风险评审记录
    template: review.md
    instruction: |
      记录范围、测试和兼容性检查结果及待确认事项。
    requires:
      - specs
      - design

  # 在原 tasks 定义中仅替换 requires
  # requires:
  #   - specs
  #   - design
  #   - review
```

创建 templates/review.md，给出“结论、风险、待处理事项、评审人”等标题。上面的片段不是完整 schema；直接复制粘贴前要保持与原 artifacts 列表的缩进一致。

```
openspec schema validate team-flow
openspec new change add-theme --schema team-flow
openspec status --change add-theme --json
```

两套优先级

选 schema 名称：显式 --schema → 变更 .openspec.yaml → 项目 config.yaml → spec-driven。

解析同名 schema 的文件：项目 openspec/schemas/ → 用户数据目录 schemas/ → 内置。用 schema which 检查覆盖来源。

> review.md 存在不表示负责人真的批准。需要人工签署或测试通过才允许实现/合并时，用评审与 CI 实现；仅增加依赖文件无法形成硬门禁。

让规格和代码一起接受评审

OpenSpec 的日常工作流读写项目文件，不替你创建业务分支、提交、推送或合并。store setup 默认可初始化新 store 的 Git 仓库，这是显式设置行为，应与日常工作区分。[R15]



推荐约定：一个变更，一个负责人

变更目录跟随业务分支，PR 同时包含计划、代码、测试与必要文档。评审按 proposal → specs → design/tasks → 代码证据顺序阅读。分工不同也保持一位负责人维护同一变更的整体一致性。

角色	重点责任
需求/产品	行为、范围、异常与验收是否符合真实需要
开发负责人	方案、任务、实现和迁移是否可执行
评审者	需求与实现对应，风险与测试证据充分
CI	固定版本、结构校验、测试与构建

归档时机选择一个并统一

团队可在合并后用跟进 PR 同步归档，也可在原 PR 内完成。前者使主规格只吸收已合并工作，但会多一次提交；后者流程简短，评审需同时看规格合并结果。归档不是“已上线”的自动证明。

两个变更修改同一需求

更新分支、读取最新主规格，比较两个 delta 的意图与已实现代码，形成完整的新需求块，再校验与测试。不要机械保留最后写入的一份。bulk-archive 可以帮助检查冲突，但业务取舍仍需负责人确认。

当一份计划需要服务多个代码仓库

Store 是单独存放 OpenSpec 规格和变更的仓库，通过本机注册后可从其他目录选用。先从单仓库起步；有跨服务计划、平台契约共享时再引入。[R16]




```
# 建立本地 store，不执行 Git 初始化
openspec store setup team-plans --path ./team-plans --no-init-git
# 注册已经克隆的 store
openspec store register ./team-plans --id team-plans
openspec list --store team-plans
openspec new change add-shared-theme --store team-plans
```

子命令	中文说明
store setup [id]	创建并注册；非交互需 id 和 --path；可写 --remote；默认 init-git
store register [path]	注册已有目录；--id；--yes 确认创建身份元数据
store list / store ls	列本机注册项
store doctor [id]	只读诊断注册、元数据与 Git 状态
store unregister <id>	取消本机注册，保留仓库文件
store remove <id>	取消注册并删除本地目录；非交互需 --yes，谨慎使用

上述命令支持 --json。注册是本机路径映射，不会自动让同事拥有相同目录。共享靠 Git：同事先克隆，再注册同一逻辑 ID。OpenSpec 不自动 clone、pull 或 push。Beta 的命令、格式与 JSON 可能变化，团队应固定版本。

选择写入位置，与提供参考资料是两件事

```
# 方案 A：本仓库有自己的规格，只引用平台规格
schema: spec-driven
references:
  - platform-contracts

# 方案 B：本仓库只保留配置，规划放外部 store
# store: team-plans
```

references 提供只读能力索引和读取命令，不把规格全文复制进提示，不改变写入根目录。store: 是配置型仓库的默认规划位置，不应与真实本地 specs/changes/ 混用。[R16]

写到哪里：解析顺序

显式 --store → 本地规划根目录 → config-only 仓库的 store: → 本机 defaultStore。存在本地规划目录时，store: 不会覆盖它。写入前检查命令输出 root 信息；无法解析显式 store 时应修正注册，不要悄悄退回本地。

```
openspec doctor --json
openspec context --json
openspec context --code-workspace planning.code-workspace
```

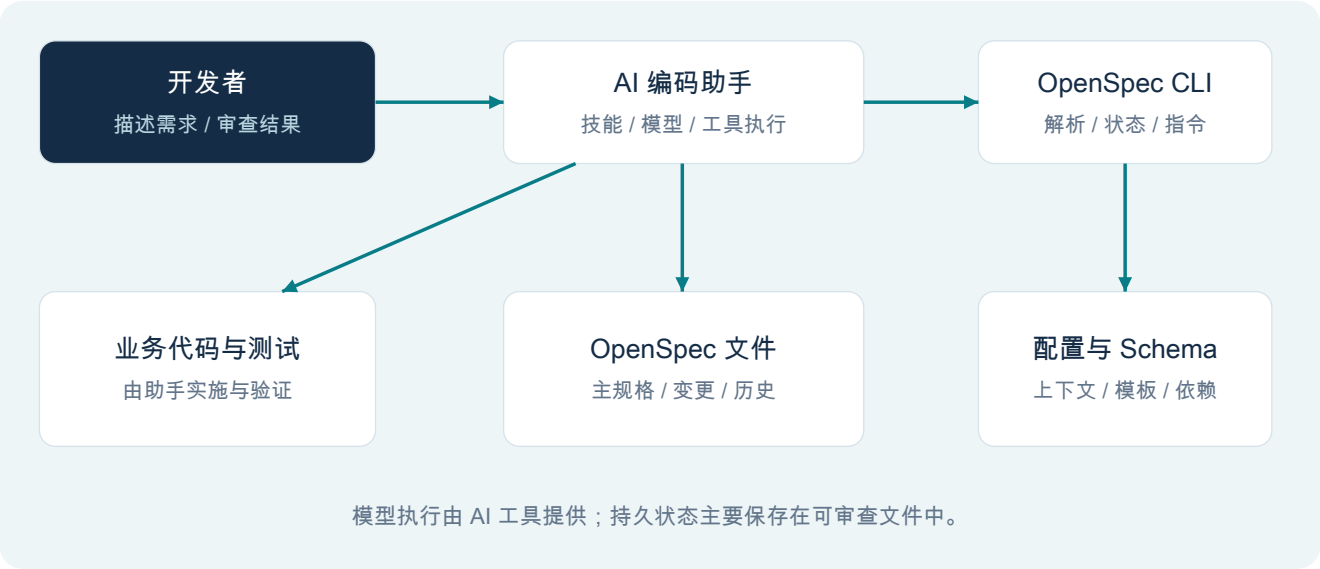
doctor 只诊断关系健康；健康报告有发现也可能退出 0，自动化应读取 status 内容。context 展示根目录与引用仓库的工作集合，--code-workspace 才会写工作区文件，覆盖已有文件需 --force。

Workset：个人常用文件夹组合（Beta）

```
openspec workset create theme-work --member ./team-plans --member ./web-app --tool codex
openspec workset list --json
openspec workset open theme-work
openspec workset remove theme-work
```

create 的第一个 member 是主要目录；open 在编辑器中打开多文件夹，或启动附带这些目录的 CLI 助手会话。--tool 可覆盖已保存工具。remove 只移除本机视图，非交互需 --yes，不删除成员仓库。Workset 不提交、不共享、不根据 references 自动推导。[R17]

人、助手、CLI 与文件如何配合



图为依据官方命令契约整理的逻辑架构，展示职责与数据流，并非官方进程部署图。OpenSpec 不内置一个代替所有助手的模型服务；模型调用和编码执行由你选用的 AI 工具完成。[R6][R12][R13]

一次 propose 的执行链

1. 工具加载生成的技能/命令，知道要按 OpenSpec 流程处理请求。
2. 助手通过 CLI 解析项目或 store 根目录、schema 和产物状态。
3. instructions 将模板、上下文、规则、依赖等输入交给助手。
4. 助手读取已有代码/规格，按依赖写出 proposal、delta、design/tasks。
5. 用户审查；apply 继续读取产物和任务，修改业务代码并验证。
6. sync/archive 把已确认行为合并进主规格，保留历史变更。

想读源码，从这些职责入手

src/cli/index.ts：CLI 命令入口与参数。

src/commands/workflow/：status、instructions、new 等工作流支撑。

schemas/spec-driven/schema.yaml：默认产物依赖、模板与指导。

src/core/：初始化、工具适配、归档和规格相关能力。

先对照一次真实命令输出，再跟到对应实现，避免从整个仓库逐文件阅读。

先定位失败发生在哪一层

现象	优先检查	处理方向
openspec 找不到	PATH 与安装位置	新开终端；which -a / where.exe；确认实际副本
npm 安装后报 Node 错误	node --version	升到 ≥20.19.0，或改独立版本
Bun 安装成功却 env: node 错误	系统 Node 是否可见	补 Node，或改独立版本
macOS 手动下载不能运行	执行权限、来源与隔离属性	chmod；核对可信来源后处理 quarantine
/opsx 指令不识别	工具的实际调用语法与 profile	查看 init 提示；重启工具；运行 update
Codex 找不到 /opsx:propose	是否使用了通用名称	使用 $openspec-propose，检查 .agents/skills/
没有 verify / continue	当前默认是 core	config profile 勾选扩展项，再 update
update 提示最新但缺新功能	当前运行 CLI 版本/安装来源	version；升级正确副本，再刷新项目
JSON 前混入其他文本	调用方式与输出流	检查 stdout/stderr；不要把整段终端文本当 JSON

一个低风险的诊断顺序

```
openspec --version
openspec version
openspec config list
openspec update --dry-run
openspec list
openspec status --all --json
```

只有确认目录、来源和计划后，才实际 update。不要先用 --force 清理问题现场。若没有 OpenSpec 项目，应在目标项目根目录 init；不要误初始化用户主目录或临时错误目录。[R5][R12]

没有计划，apply 被阻塞

查看 status --change <名称> 的 missingDeps，生成缺少的产物，或检查 schema 对条件省略的定义。不要把空文件或空复选框列表当作完成计划。

结构通过以后，还要检查语义

现象	原因与处理
没识别到场景	使用 #### Scenario:；需求使用 ### Requirement:；不要翻译这些结构关键字
需求缺 SHALL/MUST	写清强制行为；将“合理处理”改为明确可观察的结果
MODIFIED 找不到目标	主规格能力路径或需求名称错误；先 list --specs 与 show
修改后丢失旧场景	MODIFIED 写了局部片段；从基线复制完整块后再改
ADDED 与主规格重名	可能是修改需求，重新判断 ADDED/MODIFIED
无 delta 校验失败	行为有变化就补规格；纯重构显式 skip_specs: true
status 显示完成但功能没实现	状态指计划产物；看 tasks 并检查测试与代码
归档无终端报确认失败	确认效果后用 archive <名称> --yes；脚本必须指定目标
删除能力最后一条需求失败	需要 retire_capabilities: true；检查非需求内容的保留要求
已归档却主规格未更新	是否跳过了 specs；检查同步摘要、实际根目录和 delta
store 无法解析	本机未注册、路径移动或指针有误；store list/doctor 检查

安全处理退役能力

REMOVED 删除最后一条需求会移除主 spec.md，必须在变更元数据中明确 retire_capabilities: true。该文件有额外非标准内容时可能拒绝退役，先按错误信息决定内容归属。不要通过 --no-validate 隐藏需要保留的知识。[R10][R12]

回到一致状态

先检查版本管理差异，确认当前主规格、活跃 delta、归档目录和代码各处于什么状态。选择补齐实现、修订 delta 或回退未确认修改，再运行严格校验。对失败的 store 操作检查实际 root，不在另一个仓库修“同名文件”。

> validate 是结构与部分 delta 一致性检查，verify 是助手语义核对，项目测试验证行为。三者互补；任何一项单独通过都不足以证明全部需求已交付。

用最少的规则保住最关键的质量

新手日常清单

1. 先读取现有能力，避免重新发明同名或近义规格。

2. 每个 change 保持单一意图，先审规格再让助手实现。

3. 任务写出“怎么证明完成”，测试与相关实现一起完成。

4. 需求变了就更新文档；换会话时明确变更名，由文件恢复上下文。

5. 归档前检查代码、测试、任务、delta 是否一致，归档后看主规格。

团队建议：CI 的基础命令

```
# CI 环境固定 CLI 版本后运行
npm install -g @fission-ai/openspec@1.15.0
openspec validate --all --strict --no-interactive
openspec validate --archived --no-interactive
# 再运行该项目自己的测试、构建与必要静态检查
```

Node 安装法需要 Node ≥20.19.0；独立二进制 CI 可改用固定发布资产。跨仓库 CI 还需显式克隆并注册所依赖的 store，固定对应提交；仅声明 references 不会下载资料。

逐步推广，避免仪式过重

第一周让一个小功能跑通；第二周统一配置与评审顺序；发现重复需求后再调整能力边界；跨仓库确有需要再试 Stores。不要为标点修正强行编造四份长文档。

上下文与隐私

生成配置只放必要项目事实，不含密钥与真实客户资料。匿名统计默认开启，可用 openspec config set telemetry.enabled false 关闭。OpenSpec 的统计与 AI 服务读取代码的策略是两件事，后者按实际助手及组织配置处理。[R1][R12]

> 最有效的规格能帮助下一位开发者判断“什么必须保持”。持续删掉过时假设，保留当前行为与必要的设计理由。

继续阅读与核对

本书基于 2026-10-28 核对的官方站点、官方 GitHub main 文档与 v1.15.0 本机帮助。官网与 main 会继续更新，示例输出和次要参数可能变化；Beta 内容尤其需要以所用版本为准。

[R1] 项目主页与 README

https://github.com/Fission-AI/OpenSpec

[R2] v1.15.0 发布说明

https://github.com/Fission-AI/OpenSpec/releases/tag/v1.15.0

[R3] 多平台安装、升级与卸载

https://openspec.dev/docs/installation

[R4] 官方快速入门

https://openspec.dev/docs/quickstart

[R5] 项目初始化

https://openspec.dev/docs/setup

[R6] 十二个助手工作流的行为契约

https://openspec.dev/docs/skills

[R7] Profiles 与 delivery

https://openspec.dev/docs/profiles

[R8] 支持工具、安装路径与调用方式

https://openspec.dev/docs/supported-tools

[R9] 默认 schema 与模板

https://github.com/Fission-AI/OpenSpec/tree/main/schemas/spec-driven

[R10] 规格写法与 delta 规则

https://github.com/Fission-AI/OpenSpec/blob/main/docs/writing-specs.md

[R11] 实战配方与纯重构示例

https://github.com/Fission-AI/OpenSpec/blob/main/docs/examples.md

核对方法

命令组与关键选项对照本机 v1.15.0 --help；实战 delta 已在隔离目录通过 validate --strict；教学函数的 3 项 Node 测试全部通过。本文不声称已在所有操作系统、全部 AI 助手或真实团队部署中逐项运行。

把这份指南当作入口

[R12] 完整 CLI 参考

https://openspec.dev/docs/cli

[R13] 项目上下文、规则与操作建议

https://openspec.dev/docs/project-config

[R14] 自定义 Schema

https://openspec.dev/docs/customize-schemas

[R15] 团队工作流

https://github.com/Fission-AI/OpenSpec/blob/main/docs/team-workflow.md

[R16] Stores（Beta）

https://openspec.dev/docs/stores

[R17] Worksets（Beta）

https://openspec.dev/docs/worksets

[R18] 上游许可

https://github.com/Fission-AI/OpenSpec/blob/main/LICENSE

英文	本书用词	记忆提示
Source of truth	事实来源 / 当前规格	现在系统承诺什么
Delta	增量规格	这次要改什么
Proposal	提案	为什么改、改多大
Design	设计	如何实现与为何这样选
Tasks	任务	具体做什么、怎样验证
Sync	同步规格	把增量合并进主规格
Archive	归档	保存完成变更的历史
Store	规格仓库	跨代码仓库共享规划
Workset	个人工作集合	重开常用的一组目录

许可与归属

OpenSpec 上游代码与文档采用 MIT License，Copyright (c) 2024 OpenSpec Contributors。本书为独立中文编写与教学整理，引用的名称、命令及文档结构归其原作者；下页保留上游 MIT 许可全文。本书中的流程图为依据公开契约绘制的教学图，实战与团队建议已在正文标明。

上游文档许可

以下保留 OpenSpec 官方 LICENSE 的英文全文。它适用于上游软件及其相关文档的使用与再分发。[R18]

MIT License

Copyright (c) 2024 OpenSpec Contributors

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

最后一次官方资料核对：2026-10-28。建议在升级前运行 openspec version、阅读对应 Release Notes，并先用 openspec update --dry-run 检查项目集成变化。